AI201 PA 2 Report
Naive Bayes Spam Filter

Deomary Angelo Franco (2023-22049)

Date of Submission: March 12, 2025

1. INTRODUCTION

Emails play an important role in modern day communication. Everyday, different crucial communications are received via emails, either related to personal life or career. However, it is not uncommon to receive emails with malicious intent from unknown senders hence it is essential to review and understand the contents of the emails since engaging with them such as clicking links or responding will do us harm. Ideally, reading our daily emails should be quick and effortless but because of the risk of falling victim to these fraudulent emails, being extra careful is a necessity. With this, great efforts are spent to automate identifying which emails are considered spam or non-spam.

Various state-of-the-art machine learning algorithms, such as support vector machines, ensemble methods, and deep learning architectures, have already been deployed to have a robust and optimized filtering system. In this paper, a classical Naive Bayes method will be employed. A Naive Bayes classifier is derived from a Bayesian framework but with the features assumed statistically independent from one another - an approach known as a "bag of words". Although it is obvious that its words are dependent on one another, results have shown that this simplification is widely acceptable and effective.

2. OBJECTIVES

The objective is to implement a Python-based spam filtering email system using a Naive Bayes Classifier on a Jupyter notebook (.ipynb) without leveraging open-source off-the-shelf libraries, such as Scikit-Learn and NLTK, that include the algorithm.

The following are also to be performed:

- Use performance evaluation metrics such as precision and recall, then test the model on the test dataset unseen during training.
- Incorporate Lambda Smoothing to prevent numerical issues if a word in the test set does not appear during training, and analyze its effects.
- Use Mutual Information to select the top 200 most informative words and assess whether it improves the model's performance.

3. METHODOLOGY

3.1 Dataset and Preprocessing

The dataset used to train the model is the TREC06 Public Spam Corpus. The dataset contains 24,912 spam emails and 12,910 ham (where ham refers to non-spam) emails, which has approximately a ratio of 65-35. For train and test dataset split, 70-30 is employed where the dataset is split randomly but stratified according to their labels such that the ratio of spam and ham within the train set should match the ratio in the test set.

Using regular expressions, a vocabulary set will be created to store all the words used in the training dataset along with their respective counts. The words are defined as a sequence of alphabetic characters [a-zA-Z] which are separated by a space delimiter in front and either a space, comma, or period at the end. The counts are defined as the unique words per email (e.g. an email containing "the blue dog and blue woman" will only return a count of 1 for "blue").

3.2 Prediction

The Naive Bayes Classifier uses the equation below:

$$P(\omega|x_1, x_2, ..., x_d) = \frac{\prod_{i=1}^{d} P(x_i|\omega)P(\omega)}{\sum_{\omega} \prod_{i=1}^{d} P(x_i|\omega)P(\omega)} \tag{1}$$

In this implementation, ω refers to the email labels, either spam or ham, and the different words in the email are represented by x_i. The cardinality of the words within a given label is denoted by d. The equation states that the posterior probability of a given email label is equal to the normalized product of all the likelihood of each email's word and the prior probability of the email's label. The likelihood is computed as follows:

$$P(x_i|\omega) = \frac{\sum_{D \in D_\omega} I(x_i \in D)}{|D_\omega|} \tag{2}$$

The function $I(\cdot)$ is called an indicator function which returns 1 if the word exists in the email and 0 otherwise, D_ω

is the set of emails of label ω, and $|D_\omega|$ is its cardinality.

Equation 1 serves as the model that provides a guideline for predicting whether an email is spam or ham. In this implementation, the posterior probability for both labels is computed and compared, with the label having the higher posterior probability being the predicted label. However, only the numerator is used for comparison, while the denominator is ignored since it merely serves as a normalization factor. This approach assumes that our decision boundary for prediction is the intersection of both labels $P(\omega|x_1, x_2, ..., x_d) = 0.5$.

In the machine learning context, training typically means precomputing the likelihood probabilities of words in the vocabulary and have them readily available for inference. But in this implementation, a lazy evaluation is used, meaning the model only computes the likelihoods on-the-fly instead of precomputing, thereby reducing storage requirements.

Due to the large volume of the vocabulary for each class, the probabilities are very small. The product of these small values are limited by the numerical precision of the machine and would result in an underflow. To combat this limitation, the logarithm likelihood will be applied. This technique will convert the problem from the multiplication of different likelihoods of the words to the addition of the log-likelihoods of the different words. This is still valid for comparison because the logarithm function is a monotonic function (e.g. $P(x_i|\omega = spam) \geq P(x_i|\omega = ham)$ then $logP(x_i|\omega = spam) \geq logP(x_i|\omega = ham)$).

Effectively, this is how the algorithm should look like:

$$F(\omega) = \log P(\omega) + \sum_{i=1}^{d} \log P(x_i|\omega) \tag{3}$$

$$\text{Prediction} = \begin{cases} \text{spam}, & F(\text{spam}) > F(\text{ham}) \\ \text{ham}, & F(\text{spam}) < F(\text{ham}) \end{cases} \tag{4}$$

An observation here is that although the prior probability term is relatively larger compared to the individual likelihood terms $P(\omega) \gg P(x_i|\omega)$, due to the number d of words, the prior probability gets dominated by the total product of the likelihood.

3.3 Lambda Smoothing

Equation 1 is an exact representation of how to implement Naive Bayes. However, issues arise whenever the email for inference contains words not found in the training vocabulary. The likelihood from Equation 2 becomes zero and results in a zero posterior probability, which makes the prediction invalid. To combat this, Equation 2 is modified as shown below:

$$P(x_i|\omega) = \frac{\sum_{D \in D_\omega} I(x_i \in D) + \lambda}{|D_\omega| + \lambda|V|} \tag{5}$$

The added term λ serves as a hyperparameter that can be adjusted by the user. This ensures that the likelihood can never be zero. This is also one reason why lazy evaluation is the preferred design. Adjusting any precomputed likelihood probabilities is inconvenient when frequently changing the value of λ.

To determine what is the optimal λ, the model will be tested on five (5) different values. Notably, when $\lambda = 1$, this technique is commonly referred to as Laplace Smoothing.

3.4 Precision and Recall

One of the ways to measure the effectiveness of the model is to use industry-accepted metrics. Examples of these are the precision P and recall R. See below for their definitions:

$$P = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \tag{6}$$

$$R = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \tag{7}$$

The classification of a prediction as true or false is determined by comparing it to the ground truth in the test dataset. Since the sum of true positives, false positives, true negatives, and false negatives is constant, maximizing one metric often leads to a decrease in the other. The control of the trade-off usually depends on the specific usage or on what is agreed to be more important. If spam is considered positive, and if misclassifying legitimate emails as spam (False Positive) is more costly, then the precision must be maximized. Recall should be prioritized if misclassifying spam emails as legitimate (False Negative) is more costly.

In this paper, the best result would be having the best balance of precision and recall or its intersection point. The λ value from the five values which is nearest to the intersection point will be selected.

3.5 Mutual Information

To improve the model, an alternative approach is employed by reducing the vocabulary to include only the most informative words. The information provided by the words in the vocabulary is provided by the Mutual Information equation:

$$MI(x;\omega) = \sum_{x \in \{0,1\}} \sum_{\omega} P(x,\omega) \log \left(\frac{P(x,\omega)}{P(x)P(\omega)} \right) \tag{8}$$

The equation is rewritten to show likelihood terms instead of the joint probability terms.

$$MI(x;\omega) = \sum_{x \in \{0,1\}} \sum_{\omega} P(x|\omega)P(\omega) \log \left(\frac{P(x|\omega)}{P(x)} \right) \tag{9}$$

The words in the vocabulary are ranked according to their MI scores, which measure the amount of information a word provides about its associated label. The model will now only use the top 200 words as its vocabulary to compute the likelihood probabilities. The λ used here is the one selected earlier. The metrics of the new predictions will be measured and compared to the original metrics. It is important to remember that while the Mutual Information score quantifies how strongly a word is associated with its label, it does not directly indicate whether the word is more indicative of spam or ham.

4. EXPERIMENTAL RESULTS

The number of unique words, along with the prior probabilities of the training set are shown in Table 1.

The built model discussed in Subsection 3.2 and Subsection 3.3 was tested on five (5) different values of λ, the resulting precision and recall values are calculated and plotted in Figure 1. As stated earlier, the optimal choice is to have a balance of precision and recall and this is defined to be the intersection of the two. The nearest λ to this intersection is $\lambda = 0.1$. The precision and recall values are shown in Table 2 which are found to be 99.32% and 97.66% respectively.

Parameter	Value
Cardinality of Vocabulary	238396
Prior(Spam)	0.6587
Prior(Ham)	0.3413

Table 1: The number of unique words in the vocabulary parsed from the training data is 238396. The prior of spam and ham of the training set is 0.6587 and 0.3413

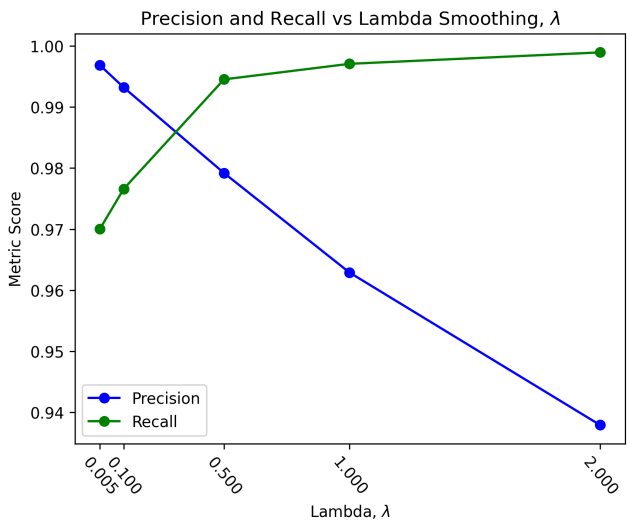

Figure 1: The precision vs. recall curve of the model for five different λ is shown. As we increase the λ value, precision decreases while recall increases, and vice versa. The value $\lambda = 0.1$ gives the point that is closest to the intersection ($\sim 98 - 99\%$), hence it is the one considered.

Vocabulary Used	Precision	Recall
Original	99.32	97.66
Top 200 most informative words	98.96	58.59

Table 2: When using only the top 200 most informative words as the vocabulary, the precision has been slightly reduced by $\sim 1\%$ and greatly reduced the recall by $\sim 39\%$. $\lambda = 0.1$ is used for both cases.

The distribution of true positives, true negatives, false positives, and false negatives can be visualized by the confusion matrix shown in Figure 2.

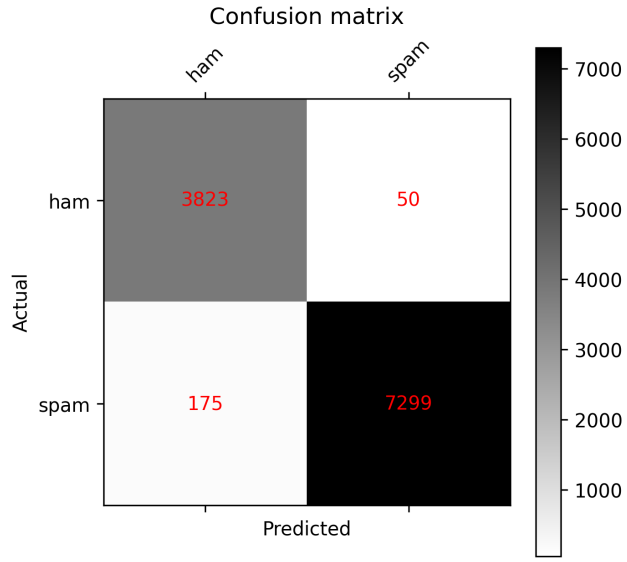

Figure 2: Shown here is a confusion matrix that shows the number of predicted and actual for spam and ham. As shown, the false positives and false negatives, 50 and 175 respectively, are small compared to the true positives and true negatives, 7299 and 3823 respectively.

By applying the approach discussed in Subsection 3.5 which involves using only the top 200 most informative words to the model with $\lambda = 0.1$, the newly calculated precision and recall values are presented in Table 2. A new confusion matrix corresponding to this reduced vocabulary is provided in Figure 3.

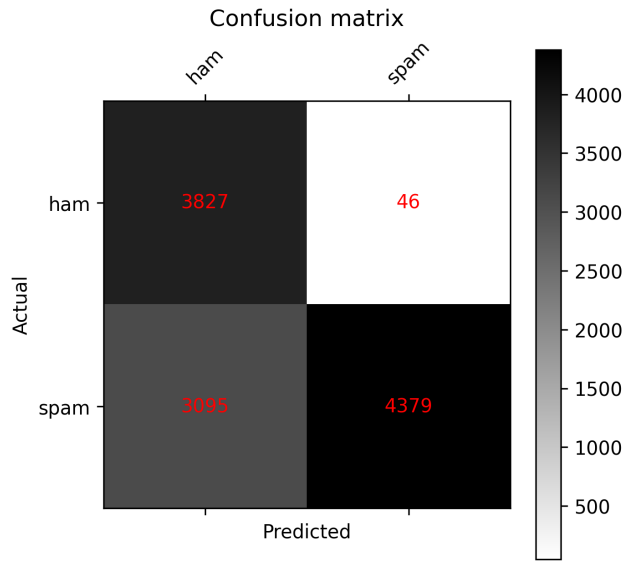

Figure 3: Upon reducing the vocabulary to only the top 200 most informative words, the model has greatly misclassified most of the spam emails to ham. This increase in False Negative significantly lowered the Recall value.

Additionally, the top 10 most informative words out of the selected top 200 are presented in Table 3.

Word	MI score
b	0.137555
unknown	0.133362
owner	0.130062
href	0.129559
br	0.113671
head	0.108710
meta	0.107797
equiv	0.106826
com	0.099307
bgcolor	0.099082

Table 3: The top 10 words from the top 200 are shown here. Most of these words are HTML tags. The presence of multiple HTML tags may greatly signify that an email is spam (or ham).

5. ANALYSIS AND DISCUSSION OF RESULTS

For the given range of $\lambda \in \{0.005, 0.1, 0.5, 1.0, 2.0\}$, the model demonstrates strong predictive performance in distinguishing if an email is spam or ham as quantified by the precision and recall scores. The precision and recall for $\lambda = 0.1$ is also impressive. Based on the confusion matrix, only a small percentage are misclassified when compared to those that have predicted correctly.

As observed, increasing λ increases the recall while decreasing precision and vice versa. The reason for this is that an increase in λ of Equation 5 causes the denominator to grow faster compared to the numerator, which in turn "numbs" the effects of the likelihood terms and improves the effect of the prior. Since our dataset has more spam, the model would likely lean towards the label with the larger prior (spam). This has a higher tendency to classify hams as spams (higher false positive and lower false negative) which results in higher recall and lower precision.

The effect of limiting the vocabulary to only the most informative words (using Mutual Information), the model has displayed a significant decline in performance scores (especially in recall) for a given lambda smoothing value. It shows that the model is underfitting and lost a handful of information. Given that the full vocabulary consists of 238,396 unique words, restricting it to only the top 200 informative words accounts for only $\sim 0.1\%$ of the total, explaining the substantial loss of predictive power.

6. CONCLUSION

This study shows that a Naive Bayes classifier can be utilized as a good spam filtering algorithm, achieving high classification performance, despite real-world emails violating the assumption of statistical independence between words. Hyperparameters such as lambda smoothing can be used to control the predictions. A higher lambda means the prediction will lean towards the dominant email label of the training dataset (in this case is "spam").

The model predicts if an email is spam or ham by using the probabilities of the appearance of the different words in the training set. However, excessively limiting the vocabulary - even to the top informative words - can lead to underfitting due to a handful of the information being lost. A potential solution is to treat the number of selected informative words as an adjustable hyperparameter, similar to the lambda, allowing for an optimal balance between underfitting and overfitting. By doing a search on the best combination of lambda smoothing value and top number of most informative words, a much better model may be determined. Just take into consideration that making the model more complex runs the risk of overfitting.

7. REFERENCES

[1] J. Hovold, "Naive bayes spam filtering using word position attributes," in *Conference on Email and Anti-Spam*. Stanford University, 2005.

[2] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A bayesian approach to filtering junk e-mail," in *Proceedings of the 1998 Workshop on Learning for Text Categorization*, AAAI. Madison, Wisconsin: AAAI Technical Report WS-98-05, 1998.